

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Einführung Konstruktoren und Operatoren

Brüche

März 2026, Version: 3.1

1 Einleitung

In dieser Aufgabe wird die Verwendung verschiedener C++-Mechanismen von **C++** geübt.

Insbesondere werden die bereits bekannten *Klassen* und *Default Parameter* verwendet und neu *Referenzen*, *const*, *überladene Funktionen* sowie *überladene Operatoren* vorgestellt.

Als Anwendungsbeispiel dient die Implementierung einer Klasse, die rationale Zahlen (Brüche mit ganzzahligem Zähler und Nenner) als Zahlentyp implementiert.

Wichtig: Bevor Sie loslegen, zunächst nochmals eine lange Liste an zum Teil schon aus der letzten Aufgabe bekannten Hinweisen. . .

Hinweis: Lesen Sie die Angabe **sorgfältig**. Lesen Sie die Angabe zu einer Teilaufgabe **komplett** durch, bevor Sie mit der Arbeit daran beginnen, um nicht wertvolle Hinweise zu verpassen.

Sollten Sie an einer Stelle nicht weiterkommen, lesen Sie die entsprechenden Themen in einem Buch oder auch den Vorlesungsunterlagen nach und **kontaktieren Sie ggf. Ihre Dozenten** der Vorlesungen und Praktika. Wir sind dafür da, Ihre Fragen zu beantworten und Fehler oder missverständliche Formulierungen in der Angabe sind nicht auszuschließen.

Verwenden Sie nicht fremde Lösungen oder ähnliche Hilfen. Das verhindert effizient, dass Ihr Gehirn zum Denken gezwungen wird und vernichtet so jeden Lernerfolg.

Hinweis: **Testen Sie** wie angegeben unbedingt nach jeden Entwicklungsschritt Ihr Programm! Dadurch vermeiden Sie eine schwer zu behebende Anhäufung von Fehlern.

Hinweis: Im Folgenden Teilabschnitt erhalten Sie Hinweise zu Fachwissen, das in der Aufgabe benötigt wird und Ihnen vielleicht noch nicht oder nicht mehr präsent ist.

Arbeiten Sie **unbedingt vor Bearbeitung** der Aufgabe (spätestens vor Beginn des betreffenden Abschnitts) diese Punkte durch, sofern nötig. Dadurch **ersparen Sie sich sehr viel Zeit** für unnötiges Herumbasteln.

Wichtig: Bevor Sie loslegen, denken Sie bitte nochmals daran: Das Ziel der Aufgabe ist nicht eine funktionierende Lösung, sondern Ihr **Lernerfolg** auf dem Weg dahin. Nutzen Sie also diesen Weg! **Seien Sie neugierig** und **klären Sie auftretende Fragen**.

Wichtig: **Dokumentieren Sie** Ihre Erkenntnisse zur Absicherung des Gelernten in Ihrem „**Laborbuch**“, wie Sie das aus PIN1 hoffentlich kennen.

Vorbereitung

- ☐ ☕☕ **Referenzen: Funktionsweise und Verwendung:** Finden Sie heraus, was Referenzen sind, was sie mit Pointern zu tun haben, wie sie sich von Pointern unterscheiden und wofür man sie sinnvoll einsetzen kann. Informieren Sie sich auch, wann und warum man das Schlüsselwort **const** für Referenzparameter verwenden sollte. 4
- ☐ ☕☕ **public und private Attribute und Methoden:** Lesen Sie nach, was **public** bzw. **private** jeweils bedeutet, wozu diese Eigenschaften verwendet werden können und was sie für Folgen haben. Überlegen Sie, was die Vor- bzw. Nachteile der Verwendung der beiden Eigenschaften jeweils sind. . . . 4
- ☐ ☕ **Konstante Elementfunktionen:** Elementfunktionen bei Klassen können **const** sein. Finden Sie heraus, was das bedeutet und wozu man dies benötigt. 4
- ☐ ☕ **Schutz gegen Mehrfachinklusion:** Klären Sie, was ein Schutz gegen Mehrfachinklusion ist und verdeutlichen Sie sich, wie er funktioniert und warum er nützlich ist. 4
- ☐ ☕ **Verwendung von ostream-Objekten für die Ausgabe:** Informieren Sie sich, wie Sie **ostream**-Objekte für die Ausgabe mit dem Ausgabeoperator << nutzen können. 4
- ☐ ☕ **Operatoren:** Von der prozeduralen Programmierung in C im ersten Semester kennen Sie bereits die Operatoren. Wiederholen Sie ggf. das Thema. 6
- ☐ ☕☕ **Überladene Operatoren:** Schlagen Sie nach, wie Operatoren überladen werden können — insbesondere wie dabei die Operanden an die Implementierung des Operators übermittelt werden und überlegen Sie sich, wozu man das Überladen der Operatoren nutzen könnte. 6

2 Grundlegende Implementierungen

2.1 Aktualisieren des Projektes



Aktualisieren Sie ihren Fork des PIN2-Projekts indem Sie **git pull upstream master** aufrufen. Überprüfen Sie anschließend, ob sich das Verzeichnis für diese Aufgabe nun in Ihrem Repository befindet.

Hinweis: Das Projekt ist in diesem Zustand noch nicht lauffähig!

5 Min

Aktualisieren Sie ihren Fork des PIN2-Projekts.

2.2 Erstellen der Klassendefinition Bruch

Bruch
— Zaehler int — Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void

Deklarieren Sie die Klasse **Bruch**, wie im nebenstehenden UML-Diagramm dargestellt, in einer neuen .h-Datei (bruch.h).

Die Methoden von Bruch sind alle public:

Name	Typ	Beschreibung
«Konstruktor»	—	Der Konstruktor dient der Zuweisung von Werten an die Attribute der Klasse. Als Übergabeparameter werden Zähler (als int mit default-Wert 0) und Nenner (als int mit default-Wert 1) übergeben.
print	void	Diese Methode gibt den Wert des Bruches auf die Konsole aus. Sie benötigt eine Referenz auf ein ostream -Objekt, auf das die Ausgabe erfolgen soll, als Übergabeparameter.

Hinweis: Die Datei bruch.h benötigt einen Schutz gegen Mehrfachinklusion.

Wichtig: Achten Sie darauf, dass das als Parameter übergebene **ostream**-Objekt beim Aufruf nicht kopiert wird.

Nach dem Deklarieren der Klasse **Komplex** in der .h-Datei werden der Konstruktor und die Methode **print** in einer neuen .cpp-Datei (bruch.cpp) implementiert.

Hinweis: Formatieren Sie die Ausgabe der Methode **print** wie für Brüche üblich, z.B.: (1/2)

Hinweis: Die Default-Werte dürfen nur **einmal** (in der .h-Datei) angegeben werden.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm sollte sich ohne Warnungen übersetzen lassen und die Ausgaben müssen mit den Vorgaben übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Hinweis: Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *unbedingt jetzt* eventuelle Fehler, bevor Sie mit der nächsten Teilaufgabe fortfahren.



Erstellen Sie die Klasse **Bruch**.

Recherche ☕☕

Referenzen: Funktionsweise und Verwendung

Recherche ☕☕

public und **private** Attribute und Methoden

Recherche ☕

Konstante Elementfunktionen

Recherche ☕

Schutz gegen Mehrfachinklusion

Recherche ☕

Verwendung von **ostream**-Objekten für die Ausgabe



Testen Sie Ihre bisherigen Implementierungen.

2.3 Zugriffsfunktionen für Lesezugriff und erweiterte Ausgabe

Bruch
– Zaehler int – Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void + print(out : ostream&, Precision : int) : void + getZaehler() : int + getNenner() : int + getValue() : double

Fügen Sie Funktionen zum Auslesen von **Zaehler** und **Nenner** sowie des Fließkommawerts mit den Namen **getZaehler**, **getNenner** und **getValue** hinzu.

Aktivieren Sie zum Testen der Ausgabe den nächsten Abschnitt in `main.cpp` durch Setzen von **testlevel** auf 1 in der entsprechenden Headerdatei und übersetzen Sie jetzt das Projekt.



Erzeugen Sie **get**-Methoden zum Auslesen von Zähler und Nenner, sowie des Gesamtwerts als Fließkommazahl.



Testen Sie die Implementierungen der **get**-Methoden mit **testlevel** 1.

Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Die Klasse `Bruch` muss in `bruch.h` um die unten beschriebene Methode ergänzt werden. Die Implementierung erfolgt in `bruch.cpp`.



Überladen Sie die Methode **print**.

Name	Typ	Beschreibung
print	void	Diese Methode gibt den Wert des Bruches als Dezimalbruch auf die Konsole aus. Hierfür benötigt die Methode zusätzlich zu einem ostream -Objekt, auf das die Ausgabe erfolgen soll, als zweiten Parameter noch die Zahl der auszugebenden Nachkommastellen (int -Wert).

Der `Bruch` soll mit dieser Methode als Dezimalzahl mit n Nachkommastellen ausgegeben werden. Da die Methode beliebig viele Stellen ausgibt, kann bei der Berechnung nicht auf die Mathematikfunktionen des Compilers zurückgegriffen werden.

Der Algorithmus zur Berechnung der n Stellen des Bruchs Z/N entspricht der in der Schule gelernten Methodik zur Berechnung von Divisionen:

1. Berechnen des ganzzahligen Anteils: Z/N und des Divisionsrests R (ganzzahlige Division der Integer-Zahlen mit Z/N ; den Rest erhält man mit Hilfe des Modulo-Operators.)
2. Berechnen der nächsten Dezimalstelle:

$$\frac{10R}{N}$$

mit Rest R für den nächsten Durchlauf.

- Der vorhergehende Schritt, die Berechnung der jeweils nächsten Dezimalstelle und des Restes R , wird so lange wiederholt, bis die n -te Stelle erreicht ist.

Aktivieren Sie zum Testen der Dezimalbruchausgabe den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 2 und übersetzen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Implementierungen von `Print` mit `testlevel` 2.

3 Überladene Operatoren für die Brüche

3.1 Der Ausgabeoperator `operator<<`

Überladen Sie den Operator `operator<<` so, dass eine Ausgabe der Form

```
cout << Bruch(12,31) << " --- " << Bruch(1,3) << endl;
```

unterstützt wird.

Hinweis: Die vorangehende Aussage ist vollgepackt mit versteckten Hinweisen:

- Damit die gezeigte Operatorverkettung funktioniert, muss der Rückgabewert der Operatorfunktion korrekt sein. Wie muss dieser aussehen?
Zerlegen Sie dazu den oben gezeigten Ausdruck und überlegen Sie, welches Ergebnis ein atomarer Teilausdruck davon haben muss, damit er als (linker) Operand bei der nächsten Ausgabeoperation im Gesamtausdruck genutzt werden kann.
- Objekte wie `Bruch(12,31)` sind konstante Objekte (genauso wie das Objekt 42).
Sie müssen also sicherstellen, dass Ihre Operatorfunktion mit konstanten Objekten als rechtem Operanden umgehen kann.

Hinweis: Sie müssen den Operator `<<` als globale Funktion überladen.

Frage: Warum muss der Operator `<<` als globale Funktion überladen werden?

Wichtig: Achten Sie bei der Implementierung unbedingt darauf, keine unnötigen Kopien des `ostream`-Objekts zu erzeugen.

Hinweis: Die Daten der Klasse `Bruch` sind zwar privat, und damit für globale Funktionen nicht zugänglich, allerdings existieren die `public-`

Recherche ☕

Operatoren

Recherche ☕☕

Überladene Operatoren



Implementieren Sie den Ausgabeoperator `<<` um Brüche auf `ostream`-Objekte ausgeben zu können.

Methoden `print` aus den vorangehenden Teilaufgaben, welche das gewünschte Verhalten realisieren können.

Aktivieren Sie zum Testen des Ausgabeoperators und der Konstruktoren die beiden nächsten Abschnitte in `main.cpp` durch Setzen von `testlevel` auf 3 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Implementierung des Ausgabeoperators mit `testlevel` 3.

3.2 Einfache Unäre Operatoren

Bruch
– Zaehler int – Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void + print(out : ostream&, Precision : int) : void + getZaehler() : int + getNenner() : int + getValue() : double + operator-() : Bruch + operator~() : Bruch

Für die praktische Arbeit mit Brüchen ist die Bildung des negativen Wertes sowie des Kehrwerts des Bruches sehr nützlich.

Da C++ keinen Operator für die Kehrwertbildung kennt, aber der Operator für die bitweise Invertierung für Brüche nicht sinnvoll anwendbar ist, verwenden wir den Operator `~` für die Kehrwertbildung¹.

Implementieren Sie die beiden unären² Operatoren - (**negatives Vorzeichen**) sowie `~` (**Kehrwertbildung**) für Brüche als Methoden der Klasse `Bruch`.

Dabei soll - einfach das Vorzeichen des Zählers umkehren, während `~` Zähler und Nenner des Bruches vertauscht.

Hinweis: Sie müssen keine Vorkehrungen für den Fall treffen, dass der Nenner des Bruches bei der Kehrwertbildung Null wird.

Aktivieren Sie zum Testen der unären Operatoren den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 4 und übersetzen und testen Sie jetzt das Projekt.

Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Implementieren Sie die unären Operatoren Negation (Vorzeichen) und Kehrbuch.



Testen Sie die Implementierung der unären Operatoren mit `testlevel` 4.

¹Bei der Kehrwertbildung tauschen Zähler und Nenner ihre Werte.

²Zur Erinnerung: Unäre Operatoren besitzen nur einen Operanden.

3.3 Arithmetische Operatoren

Bruch
– Zaehler int – Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void + print(out : ostream&, Precision : int) : void + getZaehler() : int + getNenner() : int + getValue() : double + operator-() : Bruch + operator~() : Bruch + operator*(rhs : Bruch&) : Bruch + operator*(rhs : int&) : Bruch

Als Beispiel für die diversen Rechenoperationen, welche man mit Brüchen ausführen kann, soll nun die Multiplikation implementiert werden.

Implementieren Sie zunächst den entsprechenden arithmetischen Operator für die multiplikative Verknüpfung zweier Brüche als Elementfunktion in der Klasse **Bruch**.

Manchmal möchte man aber Brüche auch mit gewöhnlichen ganzen Zahlen multiplizieren, ohne diese erst umwandeln zu müssen. Implementieren Sie daher den arithmetischen Operator zum Multiplizieren eines Bruchs und

einer Ganzzahl.

Dabei soll die Verknüpfung eines Bruchs mit einer **int**-Zahl unabhängig von der Reihenfolge der Operandentypen unterstützt werden.

Frage: Wie müssen Sie vorgehen, dass **int*Bruch** ebenso unterstützt wird wie **Bruch*int**?

Hinweis: Globale Funktionen können nicht auf **private**-Elemente der Objekte zugreifen.

Für diese Situation haben Sie eingangs Lesefunktionen für z.B. Zähler und Nenner implementiert!

Wichtig: Achten Sie sehr sorgfältig darauf, Referenzen und **const** für Parameter, Methoden und Rückgabewerte korrekt zu verwenden.

Aktivieren Sie zum Testen der arithmetischen Operatoren den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 5 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Implementieren Sie den arithmetischen Operator zur Multiplikation zweier Brüche.



Implementieren Sie den arithmetischen Operator zur Multiplikation eines Bruchs mit einem **int**-Wert.



Testen Sie die Implementierung der arithmetischen Operatoren mit `testlevel` 5.

3.4 Modifizierende Zuweisungsoperatoren

Implementieren Sie exemplarisch für die modifizierenden Zuweisungsoperatoren `+=`, `-=`, `*=` und `/=` den **Operator *= mit zwei Brüchen** als Operanden.

Hinweis: Sie können bei der Implementierung auf die bereits vorhandenen Operatoren und Funktionen zurückgreifen.



Implementieren Sie den Zuweisungsoperator `*=` für zwei Brüche.

Aktivieren Sie zum Testen des Zuweisungsoperators `*=` den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 6 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Implementierung der Zuweisungsoperatoren mit `testlevel 6`.

3.5 Vergleichsoperatoren



Erstellen Sie einen Vergleichsoperator für Brüche.

Bruch
— Zaehler int — Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void + print(out : ostream&, Precision : int) : void + getZaehler() : int + getNenner() : int + getValue() : double + operator-() : Bruch + operator~() : Bruch + operator*(rhs : Bruch&) : Bruch + operator*(rhs : int&) : Bruch + operator==(rhs : Bruch) : bool

Die Klasse Bruch soll nun einen Operator zum Vergleich von Brüchen erhalten.

Ergänzen Sie die beiden Dateien `bruch.h` und `bruch.cpp` entsprechend `operator==`. Diese Methode gibt bei Gleichheit der Brüche den Wert `true` zurück, ansonsten den Wert `false`.

Als Übergabeparameter wird eine Referenz auf einen Bruch übergeben. Dieser Bruch muss mit dem Wert des aktuellen Objekts verglichen werden.

Zur Realisierung des Vergleichs der Brüche sind zwei Lösungen denkbar:

- Die Brüche werden auf einen gemeinsamen Nenner erweitert und dann werden die Zähler verglichen. Für die Brüche

$$A = \frac{Z_A}{N_A} \quad \text{und} \quad B = \frac{Z_B}{N_B}$$

liegt also Gleichheit vor genau dann wenn

$$Z_A N_B = Z_B N_A$$

ist.

- Die Brüche werden als Fließkommazahlen berechnet und diese dann verglichen.

Hinweis: Die erste Variante vermeidet mögliche numerische Probleme, welche durch unvermeidliche Rundung in der zweiten Variante auftreten können.

Hinweis: Achten Sie bei der ersten Variante auf die Verwendung ausreichend großer Variablen.

Aktivieren Sie zum Testen der Vergleichsmethoden den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 7 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Implementierungen der Vergleichsmethoden mit `testlevel 7`.

3.6 Pre- und Postincrement

Bruch
– Zaehler int – Nenner int
+ Bruch(Zaehler : int, Nenner : int) + print(out : ostream&) : void + print(out : ostream&, Precision : int) : void + getZaehler() : int + getNenner() : int + getValue() : double + operator-() : Bruch + operator~() : Bruch + operator*(rhs : Bruch&) : Bruch + operator*(rhs : int&) : Bruch + operator==(rhs : Bruch) : bool + operator++() : Bruch& + operator++(: int) : Bruch

Implementieren Sie als nächstes exemplarisch die Pre- und Post- Inkrement-Operatoren.



Implementieren Sie die unären Operatoren Pre- und Postinkrement.

Hinweis: Die Pre- und Post-Varianten verwenden jeweils das selbe Operatorsymbol und unterscheiden sich lediglich durch die Position des Operanden (links oder rechts der Operatorposition).

Um die beiden ansonsten identischen Operatorfunktionen unterscheidbar zu machen, besitzt die Operatorfunktion des Post-Inkrementoperators ein zusätzliches Argument des Typs `int`, das ignoriert wird.

Aktivieren Sie zum Testen der unären Inkrement- und Dekrementoperatoren den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 8 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie die Implementierung der unären Inkrement- und Dekrement Operatoren mit `testlevel 8`.

4 Brüche kürzen

Bei der Durchführung von arithmetischen Rechenoperationen auf Brüchen kommt es in der Regel dazu, dass die Brüche nicht mehr in gekürzter Form vorliegen. Daher ist eine Methode nötig, welche die Brüche kürzt und sie wieder in die Normalform umwandelt.

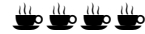
Implementieren Sie eine Methode `normalize` ohne Rückgabewert, welche Zähler und Nenner durch den größten gemeinsamen Teiler dividiert und damit die Normalform des Bruches herstellt.

Um die Brüche zu kürzen benötigen wir den Wert des größten gemeinsamen Teilers (ggT). Hierfür können Sie den Algorithmus von Euklid (siehe z.B. unter „Euklidischer Algorithmus“ bei Wikipedia) oder den optimierten Algorithmus von Stein (siehe „Steinscher Algorithmus“ bei Wikipedia) verwenden. Es bietet sich an, die ggT-Berechnung in eine eigene Methode auszulagern.

Aktivieren Sie zum Testen des Kürzens der Brüche den nächsten Abschnitt in `main.cpp` durch Setzen von `testlevel` auf 9 und übersetzen und testen Sie jetzt das Projekt. Die Ausgaben müssen alle übereinstimmen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Implementieren Sie eine Methode `normalize` zum Kürzen von Brüchen.



Testen Sie die Implementierung des Kürzens der Brüche mit `testlevel 9`.

5 Zeitlicher Ablauf

Für das Verständnis der Vorgänge kann es hilfreich sein, sich den zeitlichen Ablauf eines Programms zu vergegenwärtigen. Durch die effizienten Mechanismen der Objektorientierung, kann dieser schnell äußerst komplex werden.

In dieser Teilaufgabe sollen Sie daher an einem einfachen Fall lernen, die zeitliche Abfolge der abgearbeiteten Funktionen zu analysieren und in einem UML-Sequenzdiagramm darzustellen.

Betrachten Sie hierfür die Zeile 141 in `main.cpp`. Sie lautet

```
c *= Bruch(1,2);
```

und soll nun mittels des Debuggers untersucht werden. Setzen Sie hierfür einen Breakpoint auf diese Zeile und starten Sie den Debugger.

Arbeiten Sie sich nun mittels der Debuggerfunktionalität „step into“ durch das Programm und zeichnen Sie dabei auf, welche Funktionen dabei ausgeführt werden. Sie werden feststellen, dass:

- Die Ausführung mehrfach zwischen `main` und anderen Funktionen wechselt;



Analysieren Sie den zeitlichen Ablauf der Abarbeitung einer Anweisung und dokumentieren Sie das Ergebnis in einem UML-Sequenzdiagramm.

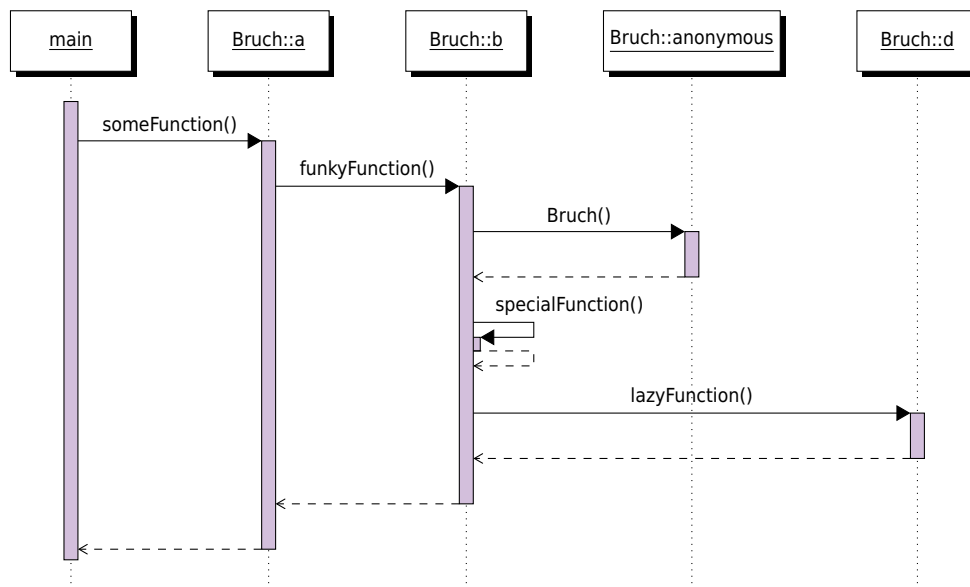


Abbildung 1: Beispiel eines UML-Sequenzdiagramms. Oben, von links nach rechts sind die beteiligten Objekte mit ihren Typen dargestellt. Nach unten läuft die fortschreitende (Programmabarbeitungs-)Zeit ab. Die durchgezogenen Pfeile stellen Aufrufe von Funktionen dar, während die gestrichelten Pfeile die Rückkehr der Funktion (also ihr Ende) signalisieren. Die länglichen Rechtecke bezeichnen die Laufzeit der einzelnen Funktionen.

- Elementfunktionen von (evtl. unterschiedlichen) Objekten aufgerufen werden; Dabei gibt es Objekte, die Variablennamen haben, aber auch solche, die anonym sind und ohne Namen vom Compiler gehandhabt werden.

Dokumentieren Sie Ihre Ergebnisse in einem UML-Sequenzdiagramm (siehe u.a. im Skript). Dieser Diagrammtyp stellt nach rechts unterschiedliche Objekte dar während nach unten die Zeit abläuft und Interaktionen (Funktionsaufrufe) durch Pfeile dargestellt werden. Abbildung 1 zeigt ein exemplarisches (und völlig sinnlos frei erfundenes) Beispiel.

Die Interpretation des Diagramms in Abbildung 1 würde folgendermaßen aussehen:

1. In der Funktion `main`, die in keinem Objekt liegt, wird die Funktion `someFunction` des Objekts `a`, das vom Typ `Bruch` ist, aufgerufen.
2. In der Funktion `someFunction` des Objekts `a` wird zunächst die Funktion `funkyFunction` eines weiteren Objekts der Klasse `Bruch` namens `b` aufgerufen.
3. In der Funktion `funkyFunction` des Objekts `b` wird zunächst die Funktion `Bruch` (also der Konstruktor) eines namenlosen Objekts der Klasse `Bruch` aufgerufen.
4. Danach wird ebenfalls von `funkyFunction` eine Funktion `specialFunction` aufgerufen, die im selben Objekt `b` liegt, wie die aufrufende Funktion `funkyFunction`.

5. Danach ruft `funkyFunction` die Funktion `lazyFunction` des Objekts `d` auf, das ebenfalls vom Typ `Bruch` ist.
6. Danach beenden sich alle noch übrigen verschachtelten Funktionen der Reihe nach und die Ausführung kann in `main` fortgesetzt werden.

Bringen Sie das von Ihnen erstellte Sequenzdiagramm unbedingt zur Abgabe mit.